

IO 流概述

集合的顶层都是接口，IO 流的顶层都是抽象类

IO 流是用来处理设备之间的 **数据传输**

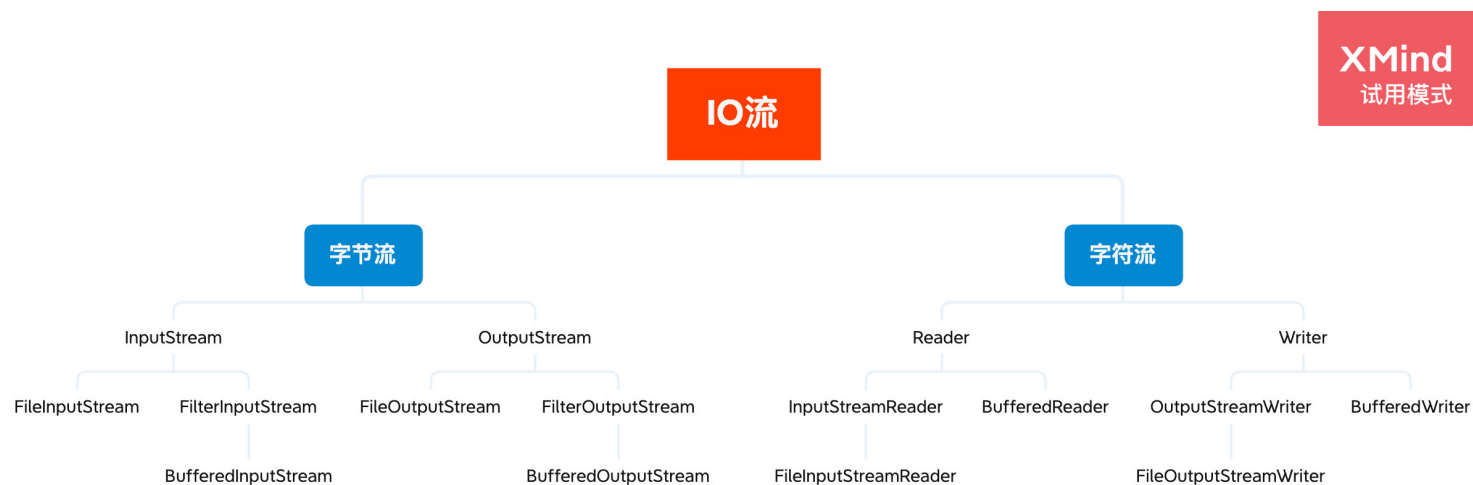
- 比如硬盘和内存间的数据传输
- 流对象尽量晚开早关，会占用资源

按流向分： **输入流**、**输出流**

按操作类型分： **字节流**（可以操作任何数据）、**字符流**（只能操作字符数据）

- 字节流抽象父类：InputStream、OutputStream
- 字符流抽象父类：Reader、Writer

中文名的文件，创建 IO 流会报错：FileNotFoundException:is a directory



字节流

```
public abstract class InputStream implements Closeable
public class FileInputStream extends InputStream

public abstract class OutputStream implements Closeable,
Flushable
public class FileOutputStream extends OutputStream

public class FilterInputStream extends InputStream
public class BufferedInputStream extends FilterInputStream

public class FilterOutputStream extends OutputStream
public class BufferedOutputStream extends FilterOutputStream
```

普通字节流

FileInputStream: 会判断文件是否存在，不存在则报错

FileOutputStream: 如果没有该文件，则会创建一个；如果有，则会清空该文件；第2个参数表示续写功能

FileInputStream - 读

FileInputStream fis = new FileInputStream(file): 会判断文件是否存在, 不存在则报错

```
FileInputStream fis = new FileInputStream("xxx.txt");    // 如果没有该文件, 则会报错
int a;
while((a=fis.read())!= -1){
    System.out.println(char(a));
}
fis.close();    // 释放资源
```

读取文件 - read

int len = fis.read(): 一次读取一个字节

- len 表示: 读取到字节对应的 int 值 (前面补 0)
- **-1 表示**: 读到末尾、或是超过文件的字节大小

int len = fis.read(byte[] buff): 一次读取多个字节

- len 表示: 读取到的有效字节长度
- -1 表示: 读到末尾
- 一次读取 buff.length 长度的字节, 并将读到的字节放入 buff 中
- 读到的字节是从 buff 的头部开始替换, 剩余的不变
- 我们可以利用 **new String(buff, start, len)** 将读取到的字节, 转为 String 对象

为什么 read 读取单个字节，返回的是 int 值，而不是一个字节？

因为字节输入流可以操作任意类型的文件，比如图片音频等，这些文件底层都是以二进制形式的存储的；

如果每次读取都返回 byte，有可能在读到中间的时候遇到 11111111

- -1 在计算机中的 **补码** 就是 11111111
- 如果用 byte 接收：11111111 对应 int 类型的 -1，我们的程序是遇到 -1 就会停止不读了，后面的数据就读不到了
- 如果用 int 接收：11111111 会在其前面补上 24 个 0 凑足 4 个字节，那么 byte 类型的 -1 这个时候就变成 int 类型的 255 了，这样可以保证整个数据读完，而结束标记的 -1 就是 int 类型

FileOutputStream - 写

`FileOutputStream fos = new FileOutputStream("xxx.txt");` **如果没有该文件，则会创建一个；如果有，则会清空该文件**

`FileOutputStream fos = new FileOutputStream("xxx.txt", true);` **true 表示续写、追加，不会清空文件**

```
FileOutputStream fos = new FileOutputStream("xxx.txt");
fos.write("你好a打大方".getBytes());
fos.write("\r\n".getBytes());
```

写入文件 - write

fos.write(int b): 自动去除前 3 个 8 位, 变成 一个字节

fos.write(byte[] b): 将该 字节数组 依次写入到文件中

fos.write(byte[] b, int start, int len): 将该字节数组中的字节从第 start 位置读取, 读取 len 长度个字节到文件

高效字节流 / 缓冲字节流

也叫高效字节流、缓冲字节流: **BufferedInputStream**、**BufferedOutputStream**

- 关闭的时候, 只需要关闭高效字节流即可, 对应的普通字节流会自动关闭

高效字节流是对普通字节流进行了包装, 内置了一个 数组缓冲区 (1024*8=8192个字节, 8k)

- 数组缓冲区 降低内存到硬盘的读写次数: 可以避免硬盘、内存的频繁数据交换, 提高效率

```
BufferedInputStream(InputStream in)
```

```
BufferedInputStream(InputStream in, int size) // 指定缓冲区大小
```

```
BufferedOutputStream(OutputStream in)
```

```
BufferedOutputStream(OutputStream in, int size) // 指定缓冲区大小
```

BufferedInputStream - 读

1. 当从文件中读取一个字节时，BufferedInputStream 会 **一次性从文件中读取 8192 个字节**，存储在数组缓冲区中，并返回一个给程序
2. 程序再次读取文件时，就无需再去文件中找了，直接在数组缓冲区中获取
3. 直到数组缓冲区中所有的都被使用过，才重新从文件中读取 8192 个字节

BufferedOutputStream - 写

1. 当向文件中写出字节时，BufferedOutputStream 不会直接写到文件中，而是 **先写入到数组缓冲区中**
2. 直到缓冲区写满 8192 个字节，再将数组缓冲区中的数据一次性写到文件里，未写满不返回
3. 这里需要注意，写入完毕，一定要调用 **flush** 方法，将缓冲区中的剩余数据释放出来，否则会导致写入的数据不全

缓冲思想 - 装饰设计模型

高效字节流使用到了 **装饰设计模式**

1. 字节流一次读取一个数组的效率，明显比一次读取一个字节的效率快很多
2. 这是加入了数组这样的缓冲区，JAVA 本身在设计的时候
3. 也是考虑到了这样的思想（装饰设计模式），所以提供了缓冲区字节流
4. 本质是：降低内存到硬盘的读写次数

字节流拷贝

字节流拷贝有4种方式：

- 逐个拷贝：效率最低
- 大数组拷贝：可能造成 OOM 异常
- **小数组拷贝**：效率高、推荐
- **高效字节流拷贝**：效率高、推荐

逐个拷贝 - 效率最低

```
FileInputStream fis = new FileInputStream("game1.png");
FileOutputStream fos = new FileOutputStream("game2.png");

int a;
while((a=fis.read())!= -1){
    fos.write(a);
}

fis.close();
fos.close();
```

大数组拷贝 - 内存溢出

利用 `available` 方法一次性拷贝，内存占用大，有可能存在 **内存溢出** 的问题

```
FileInputStream fis = new FileInputStream("game1.png");
FileOutputStream fos = new FileOutputStream("game2.png");

int fl = fis.available();
byte[] arr = new byte[fl];
fis.read(arr); // 将文件上的所有字节读取到内存中
               的arr字节数组中
fos.write(arr); // 将内存中的arr字节数组一次写到了
               文件上

fis.close();
fos.close();
```


小数组拷贝 - 推荐

小数组一般是 **1024** 的整数倍

```
FileInputStream fis = new FileInputStream("xxx.txt");
FileOutputStream fos = new FileOutputStream("yyy.txt");

int len;
byte[] arr = new byte[1024*8];
while((len=fis.read(arr))!=-1){
    fos.write(arr,0,len);
}

fis.close();
fos.close();
```

高效字节流拷贝 - 推荐

一定要调用 **flush** 方法，释放缓存中的数据；否则拷贝的文件是不全的

- 由于内部已经有数组缓冲区了，所以这里无需再重新创建一个

```
FileInputStream fis = new FileInputStream("game1.png");
    // 创建输入流对象, 关联game1.png
FileOutputStream fos = new FileOutputStream("game2.png");
    // 创建输出流对象, 关联game2.png
BufferedInputStream bis = new BufferedInputStream(fis);
    // 创建缓冲区对象, 对输入流对象进行装饰
BufferedOutputStream bos = new BufferedOutputStream(fos);

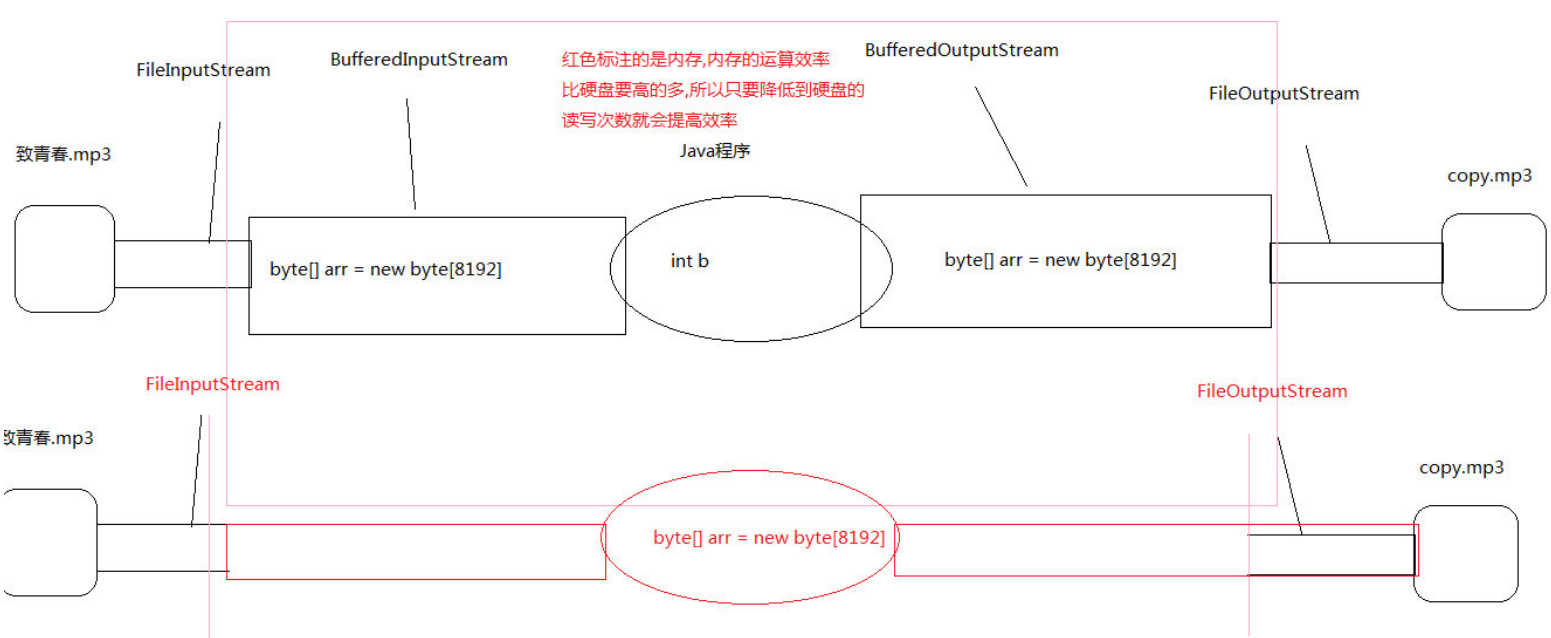
int a;
while((a=bis.read())!=-1){
    bos.write(a);
}
bos.flush();           // 重要

bis.close();           // 关闭装饰后的对象即可
bos.close();
```

小数组拷贝和高效字节流拷贝哪个效率高？

定义的小数组也是 **8192 个字节** 大小的话，和高效字节流相比，小数组会略胜一筹

- 因为小数组读写操作的是同一个数组
- 而高效字节流是两个数组：BufferedInputStream 和 BufferedOutputStream 分别占用一个数组



字符流

字符流的实现方式：读取到的字符（2 个字节）以 **int 类型** 表示

- 读：先读取到字节，判断读几个字节，然后将字节转为字符输出（补 0 输出字符，**int 类型的字符**）
- 写：先读取到字符，然后转为字节写出
- 通过编码表实现：遇到 **首个字节是负数（表示中文）**，读取 2 个字节；正数（表示英文），读取 1 个字节
- 字符流读取的也是字节，只是他会将字节拼接、转化为字符，然后输出为 **int 值**

字节流、字符流的适用范围

- 字符流：文本文件读写、拷贝（不会出现半个中文字符，可以

直接将字符串写出)

- 字节流：文本文件、非文本文件读写、拷贝（可能会出现，只读取了半个中文字符的情况）

文本文件、非文本文件的拷贝

- 文本文件的拷贝：可以用字符流，但是字符流有一个字节转化为字符的过程（int 类型的字符），效率低
- 非文本文件的拷贝：只能用字节流（在使用字符流的时候，部分字节在读的时候转为字符，可能不能被编码表识别，会用？代替，在写出时遇到？，将直接写出，导致乱码）
- 综上，拷贝的时候统一用字节流进行拷贝

字符流的用法，和对应的字节流用法基本一致

```
public abstract class Reader implements Readable, Closeable;

public class InputStreamReader extends Reader;
public class FileReader extends InputStreamReader;

public class BufferedReader extends Reader;
public class LineNumberReader extends BufferedReader;
```

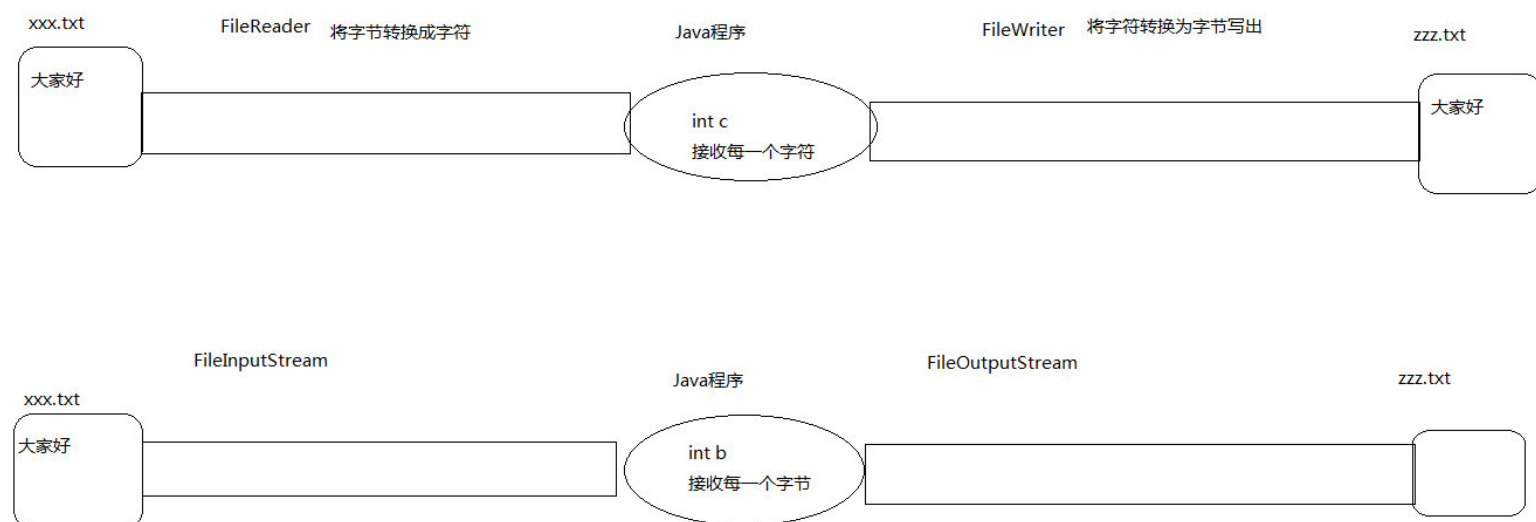
普通字符流

普通字符流：FileReader、FileWriter

- FileReader - InputStreamReader - Reader (抽象类)
- FileWriter - OutputStreamWriter - Writer (抽象类)
- FileInputStream - InputStream (抽象类)
- FileOutputStream - OutputStream (抽象类)

read、write：在进行字节流、字符流转换的时候

- 字节流：补 24 个 0，转 int 输出
- 中文字符：补 16 个 0，转 int 输出
- 英文字符：补 24 个 0，转 int 输出



FileReader

`int len = fileReader.read()`：读取一个字符

- len 表示，读取到字符的 int 值

`int len = fileReader.read(char[] buff, int start, int len)`: 读取多个字符串

- len 表示，读取到字符的长度

注意：read 的编码要和文件的编码一致，否则可能会造成乱码（Eclipse 默认是 GBK）

```
FileReader fileReader = new FileReader("abc.txt");
```

FileWriter

FileWriter 中有一个 **字符缓冲区**（ $2 \times 1024 = 2K$ ），写操作完毕，必须调用 flush、或者 close 方法，将缓冲区中的数据释放出来，否则会造成写入的数据不完整

`fileWriter.write(String str)`: 写入字符串

`fileWriter.write(String str, int start, int len)`

`fileWriter.write(char[] buff)`: 写入字符

`fileWriter.write(char[] buff, int start, int len)`

高效字符流

高效字符流：BufferedReader、BufferedWriter、LineNumberReader

- 与普通字符流相比，高效字符流可以 整行整行的读取、写入
- 字符缓冲数组：16K (16*1024K)

```
BufferedReader(Reader r)
BufferedReader(Reader r,int size)           // 指定缓冲区大小
BufferedWriter(Writer w)
BufferedWriter(Writer w,int size)           // 指定缓冲区大小

readLine()           // 读取一行文本，不包含任何终止符，遇到\r、\n或者\r\n
终止，如果已达到末尾返回null
newLine()             // 加入一个行分隔符（跨平台）
```

BufferedReader

BufferedReader 只有在遇到 `\r\n` 才会结束读取

`readLine()` 读取整行，遇到 `\r`、`\n` 或者 `\r\n` 终止，如果已达到末尾返回 `null`

BufferedWriter

BufferedWriter 不会自动输入换行符，需要我们手动调用

`newLine()` 方法写入换行符

newLine() 和 \r\n 的区别

- windows 下的换行 \r\n, Linux下的换行 \n, Mac下的换行 \r
- newLine() 可以跨平台

LineNumberReader

LineNumberReader 是 BufferedReader 的子类, 可以统计行号

- **getLineNumber()**: 获取当前行号
- **setLineNumber()**: 设置当前行号

原理: 有一个成员变量 private int lineNumber=0, 每次调用 readLine() 就会 +1

```
getLineNumber()           // 获取当前行号, 默认获取的第一个是1
setLineNumber(100)        // 设置当前行号, 设置100, 下次获取的是101
```

InputStreamReader/OutputStreamWriter

InputStreamReader、**OutputStreamWriter**: 是字符流和字节流转化的桥梁

- FileReader、FileWriter 都是用 **默认的编码表** 读写文件, 用

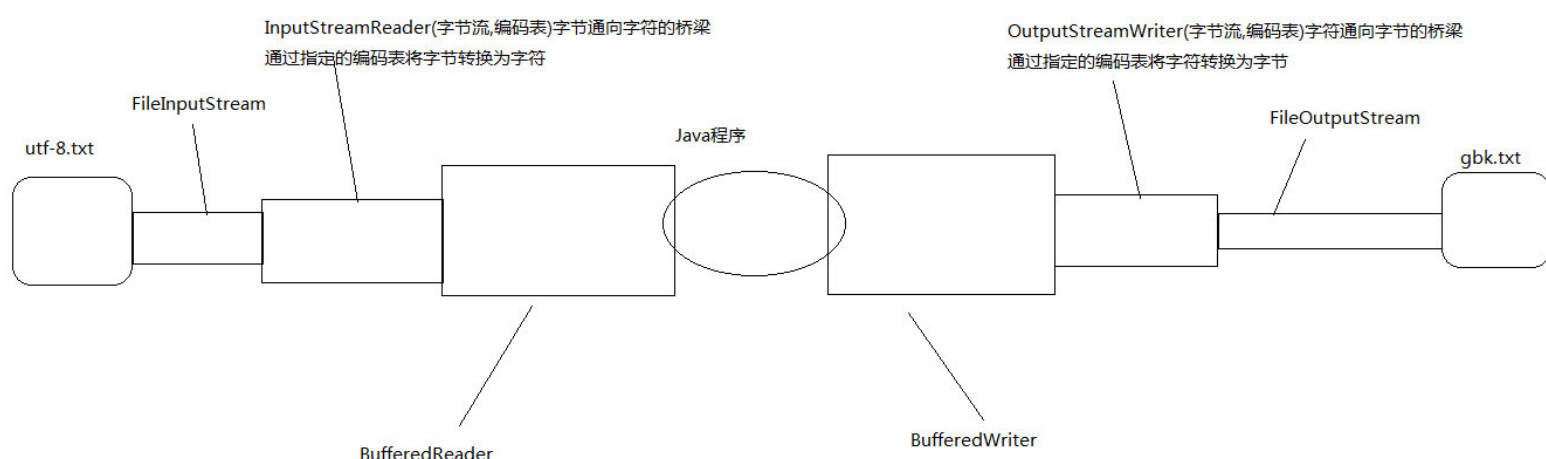
InputStreamReader、OutputStreamWriter 可以通过 指定编码表 进行转化

```
InputStreamReader(InputStream in)
InputStreamReader(InputStream in,charset)
OutputStreamWriter(OutputStream out)
OutputStreamWriter(OutputStream out,charset)
```

字节流和字符流的转换

可以用 BufferedReader、BufferedWriter 对 InputStreamReader、OutputStreamWriter 进行包装

- BufferedReader - InputStreamReader - FileInputStream



编码格式的转换

将 UTF-8 编码的文件，转为 GBK 编码的文件

```
InputStreamReader isr = new InputStreamReader(new
FileInputStream("UTF-8.txt"), "UTF-8")); // 以 UTF8 格式读取
OutputStreamWriter osw = new OutputStreamWriter(new
FileOutputStream("GBK.txt"), "GBK")); // 以 GBK 格式输出
int ch;
while((ch = isr.read()) != -1) {
    osw.write(ch);
}
br.close();
bw.close();
```

其他流

序列流 - SequenceInputStream

序列流 **SequenceInputStream**：将多个字节输入流整合成一个序列流，在读取序列流的时候，将从被整合的第一个序列流开始读，读完之后在读第二个，以此类推

```
SequenceInputStream(InputStream i1,InputStream i2)
SequenceInputStream(Enumeration<? extends InputStream> e)
```

内存操作流

内存操作流：**ByteArrayInputStream**、**ByteArrayOutputStream**

内存操作流：往内存中写入数据，将**内存当作一个缓冲区**，写完之后可以一次性读取所有的内存数据

- 可以用于读取中文字符

```
// 输入流
new ByteArrayInputStream(byte[] data);

// 输出
toString()           // 使用系统默认的编码表，将字节缓冲区的内容转变为字符串
toArray()            // 将字节缓冲区中的内容赋值到一个byte数组中
close()              // 无效，内存中的数据无需关闭

// 输出流
new ByteArrayOutputStream()
new ByteArrayOutputStream(int size)

// 写入
write(int i)
write(byte[] b, int off, int len)
```

ByteArrayInputStream

ByteArrayInputStream：从内存中读取数据

写入到内存后，我们可以调用 **toString()**、**toArray()** 将内存中的数据一次性读出

ByteArrayOutputStream

ByteArrayOutputStream: 往内存中写入数据

对象操作流

对象操作流: **ObjectInputStream**、**ObjectOutputStream**

对象操作流: 可以将一个对象写入流中, 或者将对象从流中读取, 也就是 **序列化和反序列化** 的过程 (序列化: 将对象写到文件上)

- ObjectOutputStream: 序列化, ObjectInputStream: 反序列化
- 写入的类, **必须实现 Serializable 接口**, 否则报错
NotSerializableException

对象操作流, 通常用于克隆对象、将对象写入文件中

// 序列化

```
ObjectOutputStream(OutputStream os)
writeObject(Object ob)           // 自动类型提升为Object, 读取的时候返回类型是Object
writeObject(List l)               // 可以传入一个对象集合, 读取的时候返回类型时List集合
```

// 反序列化

```
ObjectInputStream(InputStream is)
readObject()           // 返回的是Object类型, 需要进行强转
                        // 如果读取到文件末尾, 继续读, 会报错
EOFException (End Of File)
```

ObjectInputStream

ObjectInputStream: 读取对象

- Object object = ois.**readObject**()

ObjectOutputStream

ObjectOutputStream: 写入对象

- oos.**writeObject**(Object object)

克隆对象

我们可以结合 **对象操作流+内存操作流**，实现对象的克隆操作

- **普通对象（克隆后是不同的对象）**：创建一个与原对象相同的对象，但是地址值不同
- **单例对象（克隆后是不同的对象）**：要求内存中只存在一个对象，但是通过克隆的方式，可以创建二个对象，不安全
- **枚举对象（克隆后是同一个对象）**：通过枚举的方式创建一个单例类，用克隆的方式创建出的新对象，依然还是原来的那个，建议使用这种方式创建单例类

枚举类型其实就是一个单例对象，这些枚举项是在枚举类加载的时候，由 JVM 自动创建并实例化的

枚举项的创建过程是线程安全的

枚举项的构造函数是私有化的，如果通过反射调用枚举项的构造函数，会抛出异常

枚举项在序列化/反序列化的过程中，不会创建新的实例，而是通过 `readResolve()` 方法返回现有的枚举实例

```
public static void main(String[] args) throws Exception {  
    // 普通对象的赋值  
    Student s1 = new Student("lucy", 30);  
    Student s2 = new Student();  
    s2 = s1;  
    System.out.println(s1==s2); // true  
  
    // 克隆普通对象  
    Serializable s3 = clone(s1);  
    System.out.println(s1==s3); // false  
}
```

// 克隆单例对象 - Singleton是通过恶汉式创建的

```
Singleton st1 = Singleton.newInstance();
Singleton st2 = (Singleton) clone(st1);
System.out.println(st1==st2);           // false
```

// 克隆单例对象 - Single是通过Enum方式创建的

```
Single sg1 = Single.SINGLETON;
Single sg2 = (Single) clone(sg1);
System.out.println(sg1==sg2);           // true
```

```
}
```

// 克隆对象

```
private static Serializable clone(Serializable obj) throws
Exception {
    // 序列化
    ByteArrayOutputStream baos = new ByteArrayOutputStream();
    ObjectOutputStream oos = new ObjectOutputStream(baos);
    oos.writeObject(obj);
    // 反序列化
    ByteArrayInputStream bais = new
ByteArrayInputStream(baos.toByteArray());
    ObjectInputStream ois = new ObjectInputStream(bais);
    Object clone = ois.readObject();
    return (Serializable) clone;
}
```

```
public enum SingletonEnum {
```

```
    INSTANCE; // 这一行定义了枚举的一个实例，它就是SingletonEnum类的唯一实例
```

```
}
```

数据操作流

数据操作流：`DataInputStream`、`DataOutputStream`

数据操作流：可以按照基本数据类型大小读写数据

- `readInt()`、`readLong()`.....`writeInt()`、`writeLong()`.....

```
DataInputStream dataInputStream = new DataInputStream(  
    InputStream );  
DataOutputStream dataOutputStream = new DataOutputStream(  
    OutputStream );
```

打印流 - Print

打印流：`PrintStream`、`PrintWriter`

- `PrintStream`：字节流
- `PrintWriter`：字符流

我们可以通过调用打印流的 `print`、`println` 方法，非常方便的输出字符、字符串

- `print`、`println`：将字节转字符输出
- `write`：输出的是字节，通过查找对应码表打印数据


```

Person p = new Person("张三",13);
PrintStream ps = System.out;
ps.write(97); // 输出a
ps.print(97); // 输出97, 底层依赖的时
String.valueOf() -> Integer.toString()
ps.println(97); // 输出97, 底层依赖的时print +
newLine
ps.println("97"); // 输出97
ps.println(p); // 底层依赖的是String.valueOf()
-> Object.toString(), 如果是Null, 输出null

```

PrintStream

内置有一个 **字符串缓冲区**，在调用 println、输出换行符、输出 byte 数组、输出 byte 值是10（即 \r） ，才会自动刷新缓冲区，否则不会有任何的数据输出

PrintWriter

内置有一个 **字符串缓冲区**，只有在调用 println 的时候，才会自动刷新缓冲区，否则不会有任何的数据输出

标准输入输出流 - System

System 的几个流

System.out 就是一个 `PrintStream`，是标准的输出流，输出到 控制台

System.err 是一个 `PrintStream`，输出到控制台，输出红色字

System.in 是一个 `InputStream`，是标准的输入流，从 键盘 读取字节数据

- `System.in` 无需关闭，在一个程序中只有一个，一旦关闭了，就无法再创建
- `System.out` 也无需关闭，2 者都没有与硬盘上的文件产生关联，所以无需关闭

RandomAccessFile - 随机访问流

RandomAccessFile 不属于流，是 `Object` 的子类，但是它融合了 `InputStream` 和 `OutputStream` 的特点，属于 `java.io` 包下

- `rws`：表示 `rw` + 更新同步 + 文件基本信息同步
- 常用于多线程下载（**seek**、**setLength** 方法）

setLength()：创建一个指定大小的空文件（多线程下载）

seek()：在指定位置进行读写操作（常用于断点续传）

```
RandomAccessFile = new RandomAccessFile(String file, String
mode)
RandomAccessFile = new RandomAccessFile(File file, String
mode)
read()
write()
setLength()           // 创建一个指定大小的空文件
seek()                // 在指定位置读写，写的时候会替换该位置的数据
```

Properties - 属性流

Properties 表示一个 **配置文件**（持久的属性集）

- Properties 表示的配置文件通常是： **.properties 文件**
- Properties 可以读取任何文件：要求格式必须是 **key=value**、**key:value**
- 在使用 load 和 store 方法时，文件中的格式可以是 “name=张三” 或者 “name:张三”

Properties 是唯一一个可以直接和 IO 流结合使用的集合类，属于 **HashTable 的子类**

- Properties 没有指定的泛型，是通过其他方法指定泛型
- 属性列表中的每个键及其对应的值都是一个字符串
<String,String>

```
public class Properties extends Hashtable<Object, Object>;
```

常用方法

`load(InputStream in)`

`loadFromXML(InputStream in)`

`store(OutputStream out)`

`storeToXML(OutputStream out)`

`setProperty(key, value)`

- 不建议使用 `put` 和 `putAll` 方法
- `put` 和 `putAll` 方法可以传入非 `String` 类型的键值对（非安全的 `Properties` 类）应该使用 `setProperty`

```
Properties properties = new Properties();
properties.setProperty("name", "Lucy");    // 设置属性名称
```

// 从内存中加载文件

```
load(InputStream/Reader)    // 从输入流中读取属性列表
loadFromXML(InputStream)    // 从XML中读取属性列表
```

// 输出到内存中

```
store(OutputStream/Writer, comments)    // 将属性列表写到输出流
中
storeToXML(OutputStream, comments)    // 将属性列表写到输出
XML中
```

Properties 文件

不能出现中文，如果出现中文，会报错

- 中文会自动变成 Unicode 表达方式，例如 \u53CD

全类名的写法：当前包名.类名

```
ClassName=com.heima.Person  
MethodName=eat
```

ResourceBundle

ResourceBundle 也可以操作一个 Properties 文件

```
ResourceBundle bundle = ResourceBundle.getBundle(String  
baseName);  
String value = bundle.getString(String key);
```

注意事项

编码问题

gbk 编码中，一个中文占 2 个字节；utf-8 编码中，一个中文占 3 个字节

字节流在读中文的时候，可能会读到半个中文，造成乱码，new
String()方法